

Jackal

271835525@qq.com | 18943181643

EDUCATION

University of Illinois Urbana-Champaign (UIUC) — M.S. in Computer Science 2020.09 – 2022.01

Coursework: Distributed Systems; Computer Networks; Computer Graphics; Natural Language Processing

University of Michigan, Ann Arbor (UMich) — B.S. in Computer Science 2017.09 – 2020.05

Coursework: Databases; Machine Learning; Operating Systems; Computer Vision; Network Design

WORK EXPERIENCE

Quantitative C++ Developer, Domestic Hedge Fund (5–15B RMB AUM) 2023.07 – Present

Distributed Factor Computation Platform (0→1, led design & implementation)

1. Designed a C++ operator library from scratch: time-series / cross-sectional / group operators, differentially optimized for backtest vs. live data characteristics; pybind wrappers let researchers run the same code in both bt and live with zero migration cost
2. Deep optimization on hot paths in high-frequency workloads: stripped abstraction overhead and shortened call chains; single-operator-call latency reduced to the sub-microsecond (hundreds-of-ns) range
3. Built distributed task scheduling on Redis from scratch, with Airflow for workflow visualization
4. End-to-end automation: factor submission → search → validation → storage → live production, fully closed-loop; running stably across 20 machines for 2+ years with peak daily workload of 1M+ factor computations

Deep Learning Data Pipeline Optimization

1. End-to-end disk → mem → GPU throughput benchmarking and bottleneck analysis; evaluated and selected among mainstream DL storage formats
2. Rewrote PyTorch async DataLoader from scratch, eliminating synchronous blocking points and improving GPU utilization

Independent Project — Live Trading System (C++/Python, Multi-Market) 2025 – Present

Built an end-to-end pipeline from L2 market data to order execution, supporting A-share and Crypto

1. A-share tick-level order book + high-frequency factor engine (C++): unified order book for SSE/SZSE; CRTP-based compile-time static dispatch for the factor pipeline; 700 factors per cross-section computed in 300ms total, full-market on a single machine
2. End-to-end low-latency messaging (C++): Gateway callback → per-channel worker threads → Aggregator → PUB, built on ZMQ (inproc/ipc/tcp) with msgpack binary encoding
3. Python OMS (multi-market abstraction): TraderService for order routing + ParentOrder base class managing order state machines; algo orders implemented as per-market subclasses; MarketGateway / TraderGateway abstract the market-data and trading adapters, with full implementations for Binance Futures and A-share (QMT)

Quantitative Developer, IDEA 2022.01 – 2023.06

Algo Trading Module Refactor

1. Independent process + shared memory + lock-free SPSC queues for IPC with the trading main process; thread pool for parallel market-data / execution-report dispatch
2. Implemented VWAP / Iceberg / IS / Sniper execution algorithms on the new framework

High-Performance C++ Operator Library

1. Built entirely in-house, replacing the first-generation Eigen-based implementation; supports time-series / cross-sectional / group operators; multi-threaded, streaming + SIMD-optimized
2. Paired with an automated factor-search system (in-house GP algorithm, multi-machine parallel)

Infrastructure Intern, ByteDance

2021.06 – 2021.12

Developed the ARP module for a DPDK user-space network stack; built a write-priority logging system on top of a lock-free SPMC ring buffer

SKILLS

- **C++17:** multi-threading / lock-free (SPSC, SPMC); template metaprogramming (CRTP, concepts, static dispatch); perf / gdb
- **Low-latency & Systems:** DPDK user-space networking; shared-memory IPC; ZMQ (inproc/ipc/tcp); msgpack
- **Python:** asyncio; numpy/pandas; pybind; xtquant / binance-connector; shm / mmap; PyTorch async DataLoader
- **Data & Storage:** HDF5; Parquet; Redis; custom binary formats
- **Ops:** Linux performance tuning; git; docker; Airflow
- **AI Agents:** using Claude Code CLI (Max) since 2025